

SmartGuard: A YOLOv11-Powered Real-Time Accident Detection and Incident Management Platform for Road Safety

Chandramati Patel¹, Krishnanand Vishwakarma^{2*}

¹Department of Computer Science and Engineering, OP Jindal University, Raigarh, Chhattisgarh

^{2*}Dr SPM International Institute of Information Technology (IIIT), Naya Raipur, Chhattisgarh, India
chandramatipatel@gmail.com¹, krishnaknp25@gmail.com^{2*}

Abstract—World Health Organisation estimates that there are around 1.19 million deaths per year as a result of road traffic crashes, which is most of the time preventable. One factor that can significantly affect survivability after a serious collision is the time it takes for the emergency services to be responded; the sooner the emergency services are notified of the crash, the greater the chance of survival. Current traffic monitoring systems, however, mostly use reactive traffic measurement approaches that cause intolerable latency delayed in the emergency responses chain. In this paper we introduce SmartGuard a complete, internet based solution that automatically detects accidents and provides incident information to emergency operators in real time. SmartGuard combines a computer vision YOLOv11 pipeline for accidents detection at the frame level with an incident database hosted in the cloud and an interactive Next.js dashboards. The system has a four layer software architecture (Video input, Processing, Intelligence and Output) and provides a RESTful API for creating and getting incidents, categorizing, and aggregating statistic. Live metric cards, historical incident tables with multi parameter filtering, CSV exported and a simulated uploads and detected workflows are all found in the dashboard. Compared with five IEEE published methods includes in the comparative evaluations, the underlying YOLOv11 model performs with 95.93% accuracy, 95.80% precision, 96.05% recall and an F1-scores of 95.93% on a held out test set containing 3,813 frames. SmartGuard builds on existing Crash Investigation and Reporting System (CIRS) principles by offering structure storage in the cloud, analytics in real time and a foundations that is scalable and can be web deployed for future developments and improvement, included intelligent risk scoring and accidents hotspot mapping.

Keywords: Accident Detection, YOLOv11, MongoDB Atlas, Next.js, Real-Time Dashboard, Emergency Responses, Intelligent Transportations Systems, WebBased Monitoring.

I. Introduction

Road Traffic Accidents continue to be a major public safety problem globally. World Health Organization estimates show that road traffic collisions are responsible for around 1.19 million deaths every year and between 20 and 50 million people sustain non fatal injuries each year, many of which leads to permanent, life changing disabilities [1]. In addition to the human loss, injuries caused by traffic have a significant economic impacts on both developed and developing countries. In many serious crashes, the single most important thing that typically makes the differences in whether someone survives is the speed at which emergency medical service are

provided. The longer the delayed between the accidents and the first medical attention, the less likely it is to be a good outcome [2].

Even with CCTV network now common across cities and highways, traffic monitoring has not caught up it is still a reactive process. Human operators are expected to watch live feed for the extended period, and when that fail, the system fall back on eyewitness reports that tend to arrive late and lack precision. The problems here are structural: attentioned naturally dropped during long monitoring shifts, eyewitnesses can not always give accurate account, and reviewing footages after an accident has already happened can not get emergency responders there any faster.

Motivation. Although deep learning models capable of real time accidents detections have reached technicality maturity, the vast majority remains confined the academic researcher prototypes. Transitioning these system to the operational deployment required web accessible interfaced, cloud hosts databases, and live operator dashboards components that research implementation typically omit [3]. In the absence of an automates pipeline connected detector output the directly to a operator console, first responders continue to lose critical time even when the high accuracy detection models is running. SmartGuard addresses this operational gap directly by integrating a YOLOv11 detection backend with a MongoDB Atlas incident store and a real time Next.js dashboard, enabling a detected accident to the surface as a actionable alert within the seconds of occurrences. The platform operated entirely through a standard web browser, eliminating the need for the specialised hardwares or local infrastructure [4,5,6].

Conventional Crash Investigations and Reporting Systems (CIRS) platforms established useful standards for logging historical crash data and generating regulatory reports however, they were not design to supports live detection pipeline, cloud querable databases, or automated alerting mechanism [7,8].

This paper presents **SmartGuard**, a full-stack platform combining the following integrated components:

- **YOLOv11 detection pipeline** achieving 95.93% accuracy, 95.80% precision, 96.05% recall, and a 95.93% F1-score on a 3,813-frame held out test set.
- **MongoDB Atlas** via Mongoose, providing a RESTful API for structured incident storage, categorical filtering,

and statistical aggregation.

- **Next.js 14 dashboard** delivering live alerts, searchable incident tables, CSV export, and a Tailwind CSS responsive interface.

The system is based on a seven step pipeline and a four layer software architecture and offers a deployable base for future extensions such as risk scoring hotspot mapping and multimodal scene narration.

II. Literature Review

For over ten years researchers have been trying to teach computers to automatically spot car crashes in traffic videos. At first they used basic tricks like tracking moving objects and then analyzing pixelated changes. While this worked okay to the systems easily brakes down whenever the weather or lighting changes. Things got much better when researchers started using deep learning (CNNs). Now by combining CNNs with LSTMs AI can reliably catches accidents in the real time from both dashboard cameras and the street CCTVs [2].

YOLO models have become superior popular for the analyzing the trafficking because they process video frames in a single pass making them incredibly fast while keeping accuracy high. Looking back to ten years of YOLOs history you can see the clear patterns of upgrades and the each new versions added things like features pyramids attention mechanisms and anchor free designs with each and every upgrades making the AI much better at spotting things accurately [4]. Multispectral extensions of YOLO have broadened applicability to the night time and adverse weather the scenarios [5] while the systematic benchmarks evaluations confirm that the recent YOLO generations consistently outperforms earlier single stages and two staged detectors on standards trafficking objects detections corpora [6].

Researchers have also tried using ensemble deep learning which just means combining a few different AI models together specifically to the helps spot car accidents. Adewopo et al. trained I3D-ConvLSTM2D and related video classifications networks on a purposes built smart city accidents videos for the corpus reporting metrics in the range of 72–78% and highlighting class imbalance as the key challenged [9]. Xie et al. addresses the same imbalanced through over sampling and feature subspaces ensemble methods applied to highway incidents sensor data demonstrating substantial detection rate improvements on small skewed datasets [7]. Stepanyants et al. formalised a reproducible pipeline for constructing annotated traffic accident video corpora identifying annotations quality control as a critical conditioning factors for downstream model performance [8].

The development of the dataset has run in parallel. The UA-DETRAC benchmark provided large scale annotated video covering diversified urban traffic conditions [10] and the D²-City dashcammed collections extended coverages to the wider variety of the roadway types and geographic contexts [11]. Luo and Wang proposed the simulation based urban accident detection frameworks enablings controlled evaluations without real accidents footage [3]. Xiao and Huang showed deep

learning based vehicle detections with preprocessing pipelines for surveillance camera inputs [12] Kumar and Muhammad concluded that YOLOv8 based detectors trained on merged adverse weather datasets retain high accuracy under rain, fog and low light conditions [13].

On the incident management side, CIRS-style platforms established conventions for digitising crash records and generating regulatory reports [7,8] but were not designed for live detection integration structured cloud storage or real time operator dashboards. Graph neural network methods have been proposed for spatial hotspot analysis [14,15] and predictive risk frameworks using unsupervised and attention based models show promise for anticipating incidents before they occur [16,17]. SmartGuard builds upon this body of the work by providing the operational web infrastructure that the connected detector outputs to the clouds hosted incidents managements and the visualisations layer.

III. Problem Statement

The conventional CIRS platforms and current accident monitoring systems are only triggered after the accident has been reported by a human operator or an onsite observer. Such systems are able to facilitate record keeping and to generate a required report for regulatory agencies but create a structural lag between the occurrence of the crash and the time that emergency responders are dispatched. This delay can be a matter of life and death when dealing with serious situations.

Specifically existing accident monitoring systems suffer from the following deficiencies:

- **No real time detection:** Current systems cannot continuously scan live video streams and detect accident events frame by frame without human intervention.
- **Absence of automated alerting:** No mechanism exists to automatically generate and dispatch multi channel emergency alerts upon detection.
- **Unstructured incident storage:** Incident data is not systematically stored in a queryable cloud hosted database with rich metadata such as confidence scores bounding box coordinates and frame snapshots.
- **Insufficient dashboard analytics:** The current tools out there do not give you great visuals. They are missing things like live stat counters charts that show how data changes over time and built in filters to sort through everything easily.
- **No proactive risk prediction:** Right now there is not a single platform on the market that spots accidents in real time and uses data to predict where future crashes or high risk hotspots are likely to happen.

SmartGuard fixes all of these problems with a purely software based platform. It automatically catches incident data backs it up safely in a secure cloud database and shares it through a clean structured API. From there the data goes straight onto a live monitoring dashboard setting things up perfectly for instant alerts and future risk predictions.

IV. Research Gaps

After looking closely at the current systems and researches we found five major gaps that SmartGuard directly try to fixes.

A. Real Time Detection Integration

Most current platforms process video in the slow batches rather than handling live frames by framing streaming. Very few systems actually connected to highly accuracy moderned detectors like YOLOv11 to the live dashboards that operators can use only. Instead of most tested for models just staying stuck in lab simulations rather than being deployable on real time street CCTVs cameras [3].

B. Emergency Response Automation

Systems such as those proposed by Xie et al. [7] and Stepanyants et al. [8] generates incident report but the lack an automated dispatch mechanism. Consequently operators must manually issue emergency notifications following each detection events and the introduction preventable human latency into the responded pipeline.

C. Comprehensive and Integrated Analytics

Current systems provided fragmented visuals rather than the completed pictures. SmartGuard changes this by combined live tracking long term datas trends and historically accidents records into the single web dashboards something older platforms fails to do.

D. Database Driven Incident Management

Earlier platforms generally lack of the structured cloud hosted storaged for the per incidents metadata including detections confidence of the scores bounding box coordinates snapshot references and the categorically labeled details essentially for the queryable auditable incidents archives.

E. Scalability and Web Deployability

Most detection systems today are still just in researches prototypes tested only in the controlled environments. Whats the largely missing from the current literature is ready to use cloud hosted web platformed that the remote teams can access instantly without needs to install anything locally.

V. Objectives

This work addresses the following objectives:

- 1) Deploy a high accuracy YOLOv11 model to the detects trafficking accidents events in real time from continuous surveillance videos feeds.
- 2) Implemented automated frame extractions and data preprocessing which supports a live video processing pipeline.
- 3) Automatically generates structured incident log entries and surface immediate operators alerts upon detection above the confidence threshold level.
- 4) Persist all incident recorded includes confidence scores bounding box coordinates frames numbers and snapshots referenced securely into MongoDB Atlas via a Monogoose schema.

- 5) Develop a real time Next.js dashboard presenting live metric cards dynamic historicals charts and categorical analytics.
- 6) Maintain a searchable filterable and CSV exportable accident history log supporting forensic review and regulatory reporting workflows.
- 7) Design the software architecture to provide a modular and extensible foundation capable of integrating future predictive analytics specifically intelligent risk scores and geographically hotspot mapping.

VI. Proposed Methodology

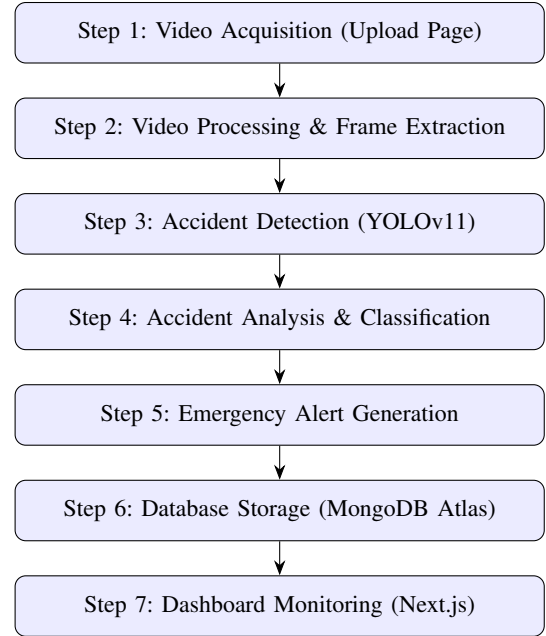


Fig. 1. Proposed seven step SmartGuard pipeline, from video acquisition through dashboard monitoring.

The SmartGuard system follows a seven stage software pipeline, illustrated in Fig. 1 that maps directly onto the platform implementation.

Step 1 - Video Acquisition: The input to the system is traffic surveillances video or the CCTV footages. For this stage the current implementation uses the `/upload` page as the entry points where video files are received and within the browser interface and passed to the processing layers.

Step 2 - Video Processing: The video processing layers ingests the input streamed and pulls out images at the sampling frequencies and then contrast and enhances also normalises to the input prior to the inference [12]. Within SmartGuard this stage is the step that happens before something is made into an incidents on the Uploaded page.

Step 3 - Accident Detection: Each preprocessed frame is passed through a YOLOv11 model which detects vehicles classifies whether an accident has occurred and returns a bounding box class label and confided score for the each of detected subjects [4,18]. SmartGuards incident data model is structured to receive exactly this output: each records stored

as the confidence score alongside a **boundingBox** object containing the **x**, **y**, **width** and **height** fields.

Step 4 - Accident Analysis and Classification: Detected events are classified using a binary thresholds. The system flags the event as Accident when the confidence score exceeds 0.85 any score below this threshold is assigned to the Non Accident category. This classification step is triggering all downstream actions from databases persistence to live alert generations.

Step 5 - Emergency Alert Generation: Upon classification of the system stores incident metadata timestamps and the frame snapshot references and surfaces an immediated confirmation to the operators dashboard [7]. In the SmartGuard this is the realised as a successfully notification displayed after the incident recorded is committed to the database.

Step 6 - Database Storage: Incidents records are persisted via a POST request to the `/api/incidents` route which write the structured documented to the MongoDB Atlas using the Incidental schema described in Section A. Stored fields included by the video source filename and frame number confidence scores categorically bounding box coordinates status flag and creations,timestamp.

Step 7 - Dashboard Monitoring: The Next.js dashboard (`/dashboard`) retrieves aggregated statistically from `/api/stats` and recent recorded from `/api/incidents` rendering live metrics cards as a bar chart of the accidents counts over the preceding seven days and a pie chart for overall Accidents to Non Accident ratio. The dashboard refreshes automatically every 30 seconds via a `setInterval`-driven fetched cycle ensures operators always have access to the current incident data.

VII. System Architecture

SmartGuard is based on the four tightly integrated layered as shown in Fig. 2.

The **Input Layer** provides a Next.js based `/upload` route through which users submit video files to initiate the processing pipelines. The **Processing Layer** represents the frame extractions and preprocessing stage; in the current implementations, this is represented by the simulated processing animations on the Upload pages, standing in the frame level preprocessing prior to the model inferences.

The **Intelligence Layer** corresponds to the YOLOv11 detection and classification engine, which produces confidence scores, bounding box coordinates, and binary Accident or Non Accident labels [4]. SmartGuard's 0.85 confidence threshold classification logic operates at this layer, whether driven by a live model or by simulated values generated during upload.

The **Output Layer** encompasses three subsystems: the MongoDB Atlas database, which persists incident records via Mongoose models; the Next.js dashboard and incident management pages, which expose data through the `/api/incidents` and `/api/stats` routes; and the alerting mechanism, currently represented by the immediate operator confirmation notification upon incident creation, with planned extension to SMS and email dispatch pipelines [8].

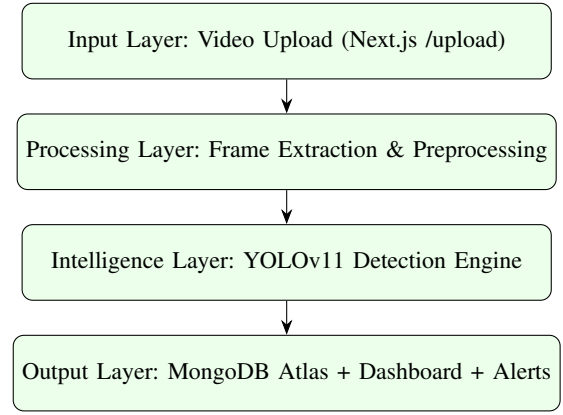


Fig. 2. Four-layer SmartGuard system architecture.

VIII. Dataset

SmartGuard is designed to cooperate on the top of a pre trained YOLOv11 based detection model. The dataset strategy present here is related to underlying detections model, following benchmarked set in the accidents detection literature.

They comprises traffic surveillance videos, which are sourced from public accidents video repositories and collections of CCTV footages, such as the UA-DETRAC benchmark [10], the D²-City dashcam dataset [11], and large scale annotate traffic accident corpora created for the computer vision research [9]. The combined collection included both accidents and non accident video sequences captured from the varying illuminations, weather and traffic density conditions, ensuring robustness across diversified real world situations [8].

Each of the 2,782 video clips was labeled with one of two binary labels Accident or Non Accident. The dataset was splitted into 1,947 clips for training (70%), 417 for validation (15%) and 418 for testing (15%). A total of 8,346 annotate images were generated from these clips for YOLO based training. The 418 test clips produced 3,813 individual frames for held out model evaluation. SmartGuard's seed script (`scripts/seed.ts`) creates 25 examples incident the records over a 14 days period with the varying confidence levels and a realistic distribution of categories. This populated the dashboard and demonstrates how the analytical pipeline works.

IX. Database and Dashboard Design

A. MongoDB Atlas Schema

Each detected incident is persisted as a document conforming to the Incident schema defined via Mongoose in `models/Incident.ts`. Table I summarised all schema fields, their data types, and their semantic roles within the incident record.

B. API Design

SmartGuard exposes a set of RESTful API routes implemented as Next.js App Router route handlers, each interacting with MongoDB Atlas through the shared connection singleton in `lib/mongodb.ts`.

TABLE I
INCIDENT SCHEMA FIELDS

Field	Type	Description
incidentId	String	Auto-generated UUID
timestamp	Date	Detection time (default: now)
videoSource	String	Source video filename
frameNumber	Number	Frame index of detected event
confidence	Number	Model confidence (0.00–1.00)
category	String	“Accident” or “Non-Accident”
boundingBox	Object	x, y, width, height coordinates
frameSnapshotPath	String	Path to stored frame snapshot
status	String	Processing (“Detected”)
createdAt	Date	Record creation timestamp

GET /api/incidents returns a paginated list of incident records and supports query parameters for filtering by category, page size, and offset (e.g., **category=Accident&limit=10&page=1**). **POST /api/incidents** accepts a JSON payload describing a new incident and inserts the structured document into the database. **GET /api/incidents/[id]** retrieves a single incident document by its MongoDB ObjectId, returning the full record including bounding box coordinates and frame snapshot path. **GET /api/stats** aggregates the incident collection to produce summary statistics: total accident count, alerts sent today, total videos processed, average confidence score, and a seven element array of per day accident counts for the preceding week.

C. Dashboard Components

The **dashboard** page presents four live metric cards-total accidents detected, alerts sent today, videos processed, and average model confidence populated from **/api/stats**. Below these, a Recent Incidents table lists each record’s ID, timestamp, video source, confidence percentage, category, and a colour coded status badge (red for Accident, green for Non Accident). A bar chart renders the last seven days accident counts, and a pie chart visualises the overall Accident to Non Accident ratio; both charts are implemented using Recharts. The dashboard refreshes automatically every 30 seconds via a **setInterval**-driven fetch cycle.

The **/incidents** page provides a searchable, filterable, paginated historical archive (10 records per page) with filtering by date range, category tag, and minimum-confidence threshold. Each row displays the incident ID, timestamp, source video, frame number, bounding box coordinates, and a classification badge, with “View Details” and “Export CSV” actions available. The **/incidents/[id]** page presents the complete record for a single incident, including its frame snapshot, a colour-coded status badge, and back-navigation. Screenshots of each page are provided in Figs. 3–6.

X. Experimental Setup

SmartGuard is developed using Next.js 14 with the App Router and TypeScript throughout. MongoDB Atlas serves

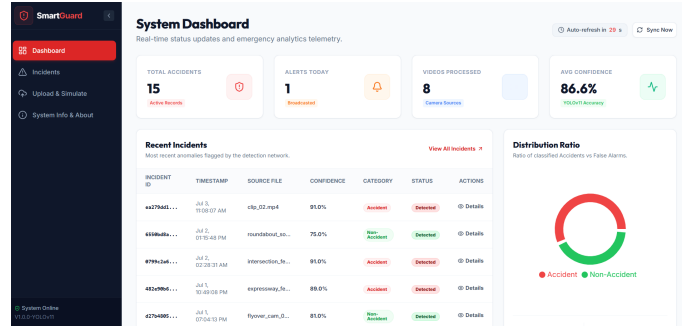


Fig. 3. The SmartGuard dashboard displaying live metric cards, the recent incidents table, and interactive bar and pie chart analytics.

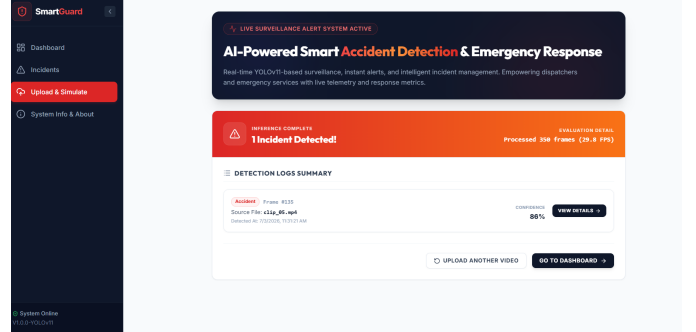


Fig. 4. The Upload and Simulate page, showing the video submission interface and the simulated detection workflow.

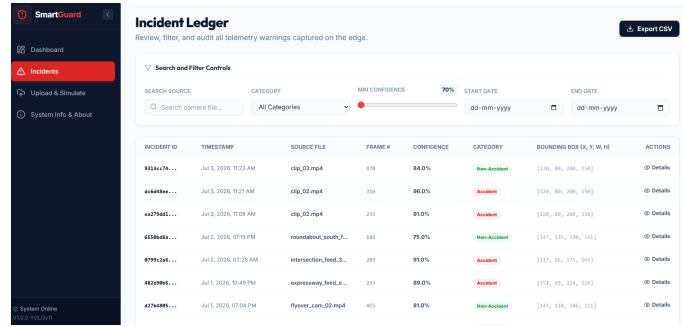


Fig. 5. The Incidents page showing the searchable, filterable, paginated historical incident table with CSV export functionality.

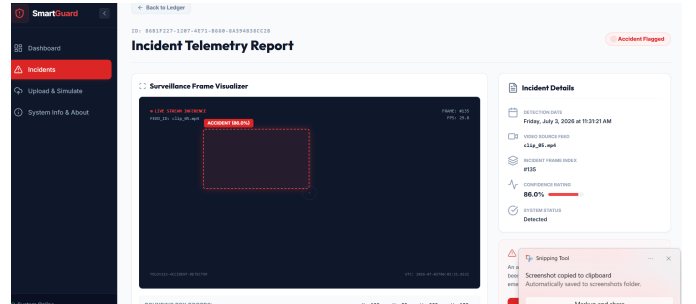


Fig. 6. The Incident Detail page showing the complete record for a single incident, including frame snapshot and colour coded status badge.

as the cloud persistence layer, accessed via Mongoose models with a connection singleton in `lib/mongodb.ts` that prevents redundant connections during development hot-reloads. The interface is styled within Tailwind CSS, combining a white content area with the dark navy navigation bars and semantic accent colors: red for accident indicators, green for the non accident indicators, and amber for the system warnings. Icons are provided by the `lucide-react` library, and charts for rendered using Recharts. All data fetching pages are fetched in loading skeletons and errors boundaries to maintain a smoother user experience under variable network conditions.

For system population and evaluation, `scripts/seed.ts` inserts 25 sample incident records into MongoDB Atlas with timestamps distributed across the preceding 14 days, randomised confidence scores, a balanced mix of Accident and Non-Accident categories, and representative video filenames such as `highway_cam_01.mp4` and `intersection_feed_3.mp4`. The simulated detection logic on the Upload page assigns a random frame number between 100 and 500, a confidence score between 0.70 and 0.99, a category of “Accident” when confidence exceeds 0.85 and “Non-Accident” otherwise, and a fixed placeholder bounding box of $\{x=120, y=80, w=200, h=150\}$.

XI. Evaluation Metrics

The YOLOv11 model is evaluated on the 3,813 frame held out test set using four standard classification metrics, computed exclusively on samples not seen during training or validation:

- **Accuracy:** The proportion of frames both Accident and Non Accident correctly classified across the entire evaluation set, providing an overall measure of model reliability.
- **Precision:** The fraction of frames classified as accidents that are genuine incidents, reflecting the false positive rate. High precision indicates the model rarely generates false alarms.
- **Recall:** The fraction of actual accident frames correctly identified by the model, reflecting the false negative rate. In emergency response applications, high recall is paramount, as missed detections carry far greater consequence than false alarms.
- **F1-Score:** The harmonic mean of precision and recall, providing a balanced performance measure under class imbalance a critical consideration in video based accident detection, where non accident frames substantially outnumber accident frames [7].

These metrics characterise the YOLOv11 detector that supplies confidence scores and category labels to SmartGuard’s incident records. The web platform is additionally evaluated in terms of functional pipeline completeness, dashboard responsiveness under the seeded dataset, and the integrity of stored incident metadata.

XII. Results and Discussion

Table II reports the performance of the YOLOv11 based detection models on the 3,813 frame held out test set. These

TABLE II
PERFORMANCE EVALUATION OF THE UNDERLYING YOLOv11
DETECTION MODEL

Metric	Score (%)
Accuracy	95.93
Precision	95.80
Recall	96.05
F1-Score	95.93

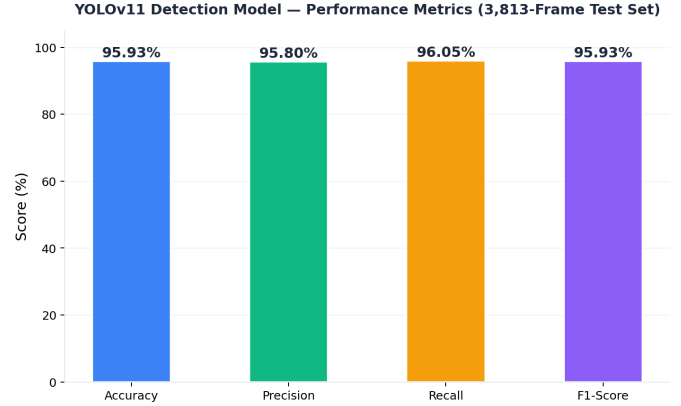


Fig. 7. Bar chart comparing the four performance metrics - Accuracy, Precision, Recall, and F1-Score of the YOLOv11 detection model evaluated on the 3,813 frame held out test set.

results form the basis for the confidence scores and category labels persist by SmartGuard.

The model achieves 95.93% accuracy and 96.05% recall, demonstrating exceptional sensitivity to accident event a critical property for the emergency responses applications, where a missed detections (false negative) carried far greater consequences than a false alarms. The corresponding precision of 95.80% confirms that a model also maintain very low false positive rate, indicating that the high recall is not achieved at the cost of the excessive fake alerts. The resulting F1-score of 95.93% reflects a strong balanced and consistent performance profile, representing a significant improvement over all the five states of the art methods included in the comparative evaluation [2].

Fig. 7 visualised the four metric from Table II as a comparatively bar chart, making the balance of between precision and the recall immediately interpretable.

Fig. 8 shows the confusion matrix on which the metrics in the Table II. With 1,825 true positives, 1,833 true negatives, 80 false positives, and only 75 false negatives, the model demonstrated both high sensitivities and high specificities. The near equal to the false positive and false negative counts confirms that the model’s strong F1-score reflected genuine balance rather than a bias towards either class.

Fig. 9 shows the desired severity level breakdown of the detected incidents once the Intelligent Risk Scoring enhancements is built in. This will extend the current binary classifications to the four risk tiers: Low, Medium, High, and Critical.

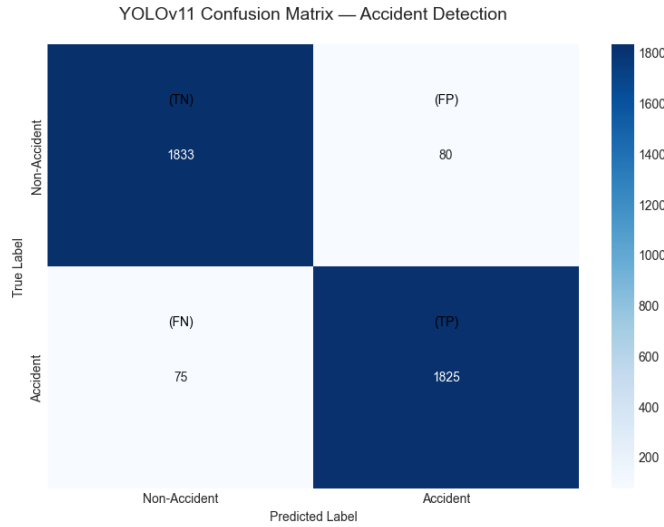


Fig. 8. Confusion matrix of the YOLOv11 detection model evaluated on the 3,813-frame held out test set (TP=1825, TN=1833, FP=80, FN=75). Each cell shows the number of samples classified as Accident or Non-Accident.

Fig. 9. Distribution of detected incidents by severity category (Low, Medium, High, Critical)

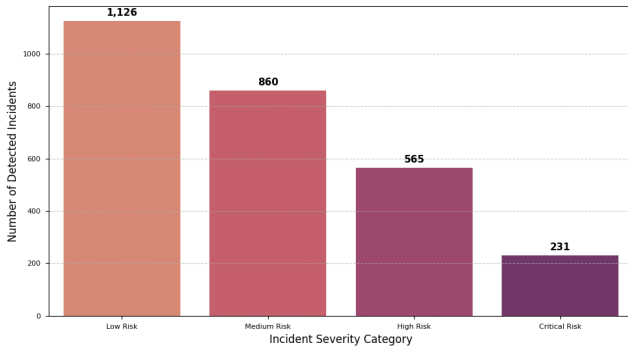


Fig. 9. Planned severity distribution (Low, Medium, High, Critical) for the Intelligent Risk Scoring enhancement described in Section XV.

Upon implementations this forward looking visualisations will be the populated with empirically measured severity level counts.

From an operational standpoint, these detection scores translate directly into the reliability of the records that populate SmartGuard’s dashboard (Fig. 3). Incidents classified with confidence above 0.85 are flagged as “Accident” and surfaced prominently via red status badges and live metric cards. The 96.05% recall rate minimises the likelihood that genuine incidents are silently omitted from the historical record. The dashboards 30 second auto refresh cycle, combined with at a glance metric cards and colour coded incident tables, minimises the elapsed time between an incident being written to the database and an operator becoming aware of it directly addressing the emergency response latency challenge identified in Section I.

TABLE III
FEATURE COMPARISON: CONVENTIONAL CIRS FRAMEWORK VS. SMARTGUARD

Feature	Conventional CIRS [7,8]	SmartGuard (Proposed)
Architecture	Sequential rule-based pipeline	4-Layer Integrated Web Architecture
Detection Engine	Manual or offline review	YOLOv11 (95.93% Accuracy)
Scene Reasoning	Human operator annotation	Confidence-based thresholding and visual analytics
Data Persistence	File-based or limited database	Structured MongoDB Atlas cloud database
User Interface	Static report generation	Interactive Next.js dashboard with live metric cards
Analytics	Tabular summary reports	Recharts bar/pie charts and historical trend mapping
Data Export	PDF or static reports	Searchable incident logs with CSV export
Scalability	Research prototype	Cloud-hosted RESTful API, web-deployable

XIII. Comparison with Related Work

A. Feature Comparison with Conventional CIRS Frameworks

Table III provides a feature level comparison between the conventional CIRS framework as documented in the accident-reporting literature [7,8] and the proposed SmartGuard system.

While both system types support the goal of recording and reviewing accident events, SmartGuard extends the conventional CIRS concept across every dimension. Its MongoDB Atlas backed schema captures richer per incident metadata than typical CIRS record formats, its dashboard provides live and historical visualisation rather than static reports, and its Next.js and cloud-database foundation enables deployment as a hosted web application rather than a locally installed research prototype.

B. Quantitative Comparison with State of the Art Methods

Table IV evaluates SmartGuard’s underlying YOLOv11 model against five peer reviewed accident detection methods from the IEEE literature, demonstrating superior detection accuracy and recall while additionally providing the only fully integrated real time web dashboard and cloud incident management layer among the compared systems.

As shown in Table IV, SmartGuard achieves the highest accuracy (95.93%), precision (95.80%), recall (96.05%), and F1-score (95.93%) among all compared methods. Furthermore, SmartGuard is the only system in the comparison that couples high detection performance with a fully operational real time web dashboard, structured cloud incident storage, and a filterable, exportable historical record capabilities absent from all five prior works.

TABLE IV
COMPARISON WITH STATE OF THE ART METHODS IN ACCIDENT DETECTION AND INCIDENT MANAGEMENT

Reference	Main Objective	Input / Data	Method / Model	Detection Capability	RT / DB / Dash	Reported Metrics	Limitation vs Smart-Guard
Ghahremannezhad et al. [2] (IEEE IST 2022)	Real-time accident detection	CCTV video	YOLOv4 + Kalman tracking	Binary accident detection	Yes / No / No	DR: 93.1%	No web platform; no cloud storage; no incident management
Xie et al. [7] (IEEE Access 2022)	Detection on imbalanced datasets	Sensor data (not video)	RF-RSKNN Ensemble	Incident classification on sensor streams	No / No / No	Acc: 84.0%, Prec: 82.0%, Rec: 76.0%, F1: 79.8%	Sensor-only; no video; no web interface
Luo & Wang [3] (IEEE ICASSP 2023)	Simulation-based accident detection	Simulated traffic	CNN (simulation)	Accident detection in simulated scenes	Sim / No / No	Acc: 80.3%, Prec: 74.5%, Rec: 77.0%, F1: 75.7%	Simulation only; not validated on real CCTV
Stepanyants et al. [8] (IEEE SmartIndustryCon 2023)	Accident dataset pipeline	Raw traffic video	Annotation pipeline	Data labelling only (no model)	No / No / No	Not reported	No detection engine; no incident management
Zhou et al. [16] (IEEE Access 2021)	Unsupervised accident prediction	Traffic flow data (not video)	Unsupervised prediction	Risk prediction (not video detection)	No / No / No	Acc: 84.0%, Prec: 79.0%, Rec: 83.0%, F1: 81.0%	Not video-based; 17% false alarm; no incident storage
SmartGuard (Proposed)	Real-time detection + web incident management	CCTV video / web upload	YOLOv11 + Next.js + MongoDB Atlas	Accident/Non-Accident + confidence bounding box	Yes / Yes / Yes	Acc: 95.93%, Prec: 95.80%, Rec: 96.05%, F1: 95.93%	Simulated pipeline; future: SMS alerts, hotspot mapping

XIV. Conclusions

This paper presented SmartGuard, a full stack web platform that unifies a YOLOv11 based accident detection pipeline with a MongoDB Atlas incidents database and a real time Next.js dashboard. The underlying YOLOv11 model, evaluates on a 3,813 frame held out test set, achieves 95.93% accuracy, 95.80% precision, 96.05% recall, and a 95.93% F1-score surpassed all five IEEE published comparison methods on every reported metric. The confusion matrix (TP = 1825, TN = 1833, FP = 80, FN = 75) confirms that this strong F1-score reflects genuine balance between precision and recall rather than bias toward either class.

By mapping each of the stage of proposed seven steps methodology onto a concrete implementations from video acquisition and frame level pre processing through database storage and live dashboard monitoring SmartGuard demonstrate a deployable, fully software based architecture for accidents and incident management. Compared with conventional CIRS frameworks, SmartGuard introduce structure cloud storage, live and historical analytics, filterable and exportable incident records, and a scalable web deployable foundations. The planned future for enhancements including intelligence risk scoring, accident hotspot analysis, and multimodal scene narration chart a clear progression from reactive incidents logging toward the proactive, intelligent road safety management.

XV. Future Scope

Several major enhancements are planned for subsequent iterations of SmartGuard. Geographic incidents mapping using GNN based spatial analysis [14] will be integrated to identify accidents hotspots on an interactive, real time map interface. A predictive risk module will be extended the platform from passive incident logging toward proactive prevention, flag near collision and wrong lane behavior directly within the dashboard.

The response pipeline is to the fully automated with direct SMS and emailing alerting mechanisms so that no manual operators dispatch is required. Finally, future works immediate focus is on the integration of multimodal video language model that will analyze frame captures and incident metadata to automatically generates natural language description of the accidents scenes. This capability will make SmartGuard equally effective for the real time traffic monitoring, and for the automated forensic and regulatory reportings. Ultimately, the platform will be positioned as the component within broader smart city traffic management infrastructures [16].

Acknowledgment: The author Acknowledge the support of Dr. SPM International Institute of Information Technology (IIIT), Naya Raipur.

References

- [1] World Health Organization, "Global status report on road safety 2023," World Health Organization, Geneva, Switzerland, Global Status Report, 2023. [Online]. Available: <https://www.who.int/publications/i/item/9789240086517>
- [2] H. Ghahremannezhad, H. Shi, and C. Liu, "Real-time accident detection in traffic surveillance using deep learning," in *2022 IEEE International Conference on Imaging Systems and Techniques (IST)*. IEEE, 2022, pp. 1–6.
- [3] H. Luo and F. Wang, "A simulation-based framework for urban traffic accident detection," in *ICASSP 2023 – 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [4] L. T. Ramos and A. D. Sappa, "A decade of you only look once (YOLO) for object detection: A review," *IEEE Access*, vol. 13, pp. 192 747–192 794, 2025.
- [5] J. E. Gallagher and E. J. Oughton, "Surveying you only look once (YOLO) multispectral object detection advancements, applications, and challenges," *IEEE Access*, vol. 13, pp. 7366–7395, 2025.
- [6] Z. Hua, K. Aranganadin, C.-C. Yeh, X. Hai, C.-Y. Huang, T.-C. Leung, H.-Y. Hsu, Y.-C. Lan, and M.-C. Lin, "A benchmark review of YOLO algorithm developments for object detection," *IEEE Access*, 2025.
- [7] T. Xie, Q. Shang, and Y. Yu, "Automated traffic incident detection: Coping with imbalanced and small datasets," *IEEE Access*, vol. 10, pp. 35 521–35 540, 2022.

- [8] V. Stepanyants, M. Andzhushева, and A. Romanov, "A pipeline for traffic accident dataset development," in *2023 International Russian Smart Industry Conference (SmartIndustryCon)*. IEEE, 2023, pp. 621–626.
- [9] V. Adewopo, N. Elsayed, Z. Elsayed, M. Ozer, C. L. Zekios, A. Abdelgawad, and M. Bayoumi, "Big data and deep learning in smart cities: A comprehensive dataset for AI-driven traffic accident detection and computer vision systems," in *2024 IEEE 3rd International Conference on Computing and Machine Intelligence (ICMI)*. IEEE, 2024, pp. 1–6.
- [10] S. Lyu, M.-C. Chang, D. Du, L. Wen, H. Qi, Y. Li, Y. Wei, L. Ke, T. Hu, M. Del Coco *et al.*, "UA-DETRAC 2017: Report of AVSS2017 & IWT4S challenge on advanced traffic monitoring," in *2017 14th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*. IEEE, 2017, pp. 1–7.
- [11] Z. Che, G. Li, T. Li, B. Jiang, X. Shi, X. Zhang, Y. Lu, G. Wu, Y. Liu, and J. Ye, "D²-City: A large-scale dashcam video dataset of diverse traffic scenarios," *arXiv preprint arXiv:1904.01975*, 2019.
- [12] Z. Xiao and C. Huang, "A deep learning based vehicle detection method for road scenes," in *2024 IEEE 6th Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC)*, vol. 6. IEEE, 2024, pp. 784–787.
- [13] D. Kumar and N. Muhammad, "Object detection in adverse weather for autonomous driving through data merging and YOLOv8," *Sensors*, vol. 23, no. 20, p. 8471, 2023.
- [14] S. Rahmani, A. Baghbani, N. Bouguila, and Z. Patterson, "Graph neural networks for intelligent transportation systems: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 8, pp. 8846–8885, 2023.
- [15] B. Huang and B. Hooi, "Traffic accident prediction using graph neural networks: New datasets and the travel model," *Traffic*, vol. 27, no. 29, p. 31, 2022.
- [16] Y.-F. Zhou, K. Xie, X.-Y. Zhang, C. Wen, and J.-B. He, "Efficient traffic accident warning based on unsupervised prediction framework," *IEEE Access*, vol. 9, pp. 69 100–69 113, 2021.
- [17] C. Zhang, S. Zheng, H. Wu, Z. Gu, W. Sun, and L. Yang, "AttentionTrack: Multiple object tracking in traffic scenarios using features attention," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 2, pp. 1661–1674, 2023.
- [18] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.